Simulador de Escalonamento
Projeto A: documentação das telas

1 Introdução

Este documento apresenta as telas do simulador de escalonamento do Projeto A, explica a função de seus elementos visuais e identifica a origem das informações exibidas. O programa simula tarefas periódicas com os algoritmos preemptivos Rate Monotonic (RM) e Earliest Deadline First (EDF), utilizando múltiplas CPUs e um histórico organizado por unidades de tempo, denominadas *ticks*.

A interface é implementada com CustomTkinter. O gráfico de Gantt é desenhado no Canvas do Tkinter, enquanto a biblioteca drawsvg produz a exportação em SVG. As figuras são capturas reais da aplicação, reaproveitadas do relatório de telas. As ampliações correspondem a recortes dessas mesmas capturas. A lista de eventos do arquivo de configuração pertence ao Projeto B e não é executada nesta implementação.

O exemplo das figuras de simulação utiliza o arquivo `configuracao.txt`, com RM, duas CPUs e três tarefas periódicas. No repositório de desenvolvimento, esse arquivo está na pasta `exemplos`; na versão preparada para envio, os exemplos estão reunidos na raiz da pasta.

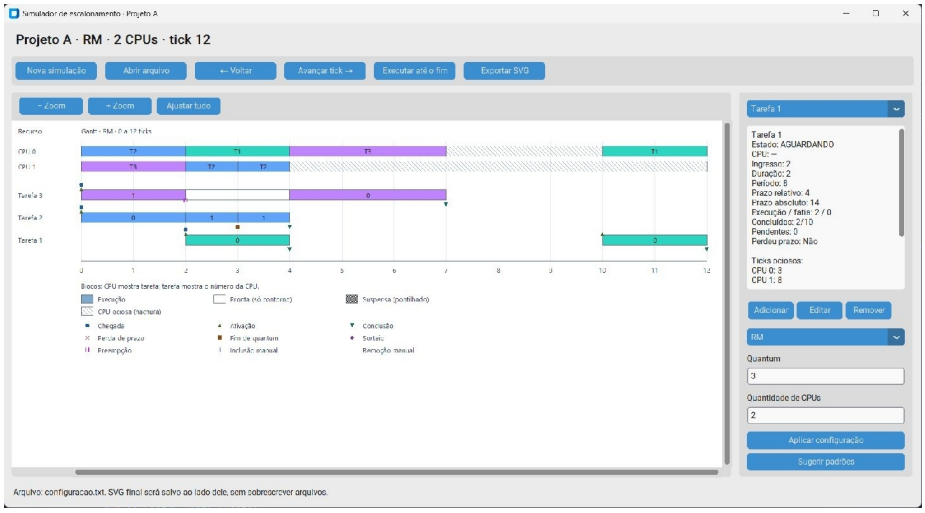


Figura 1: Janela principal da simulação no tick 12.

2 Janela principal

2.1 Tela inicial

A Figura 2 mostra a aplicação antes de carregar ou criar uma simulação. Nessa situação, o gráfico está vazio e o seletor apresenta a opção “Sem tarefas”.



Figura 2: Tela inicial, anterior ao carregamento de uma configuração.

Área do gráfico e seletor. Permanecem sem dados até que uma simulação seja criada ou carregada. O seletor lista as tarefas do estado selecionado.

Controles disponíveis. Os botões “Nova simulação” e “Abrir arquivo” iniciam o trabalho. Os demais controles dependem da existência de uma simulação.

Valores iniciais. As sugestões são quantum 0 e duas CPUs, definidas em `padroes.py`. Ainda não há uma simulação em execução.

Mensagem inferior. O rodapé orienta o usuário a abrir um arquivo ou criar uma nova simulação. O texto é definido pela interface.

2.2 Navegação e controles

A Figura 3 apresenta a barra de navegação e os controles de zoom.



Figura 3: Resumo da configuração, navegação e controles de visualização.

Título e resumo. Identificam o Projeto A e mostram algoritmo, quantidade de CPUs e tick atual. Os valores vêm de `Simulacao.algoritmo` e `estado_atual`.

Nova simulação. Cria um estado vazio no tick 0, com RM, quantum 0 e duas CPUs. As sugestões vêm de `padroes.py`; a criação é realizada pelo método `JanelaSimulador.nova`.

Abrir arquivo. Abre o seletor do Windows. O caminho escolhido é enviado a `parser.carregar_configuracao`.

Voltar e Avançar tick. Movem o índice do histórico. Avançar reutiliza um estado futuro já calculado ou calcula o próximo tick. Voltar não ultrapassa o instante zero.

Executar até o fim. Avança até que cada tarefa conclua dez ocorrências. A interface agenda lotes por `after()`, e o motor informa o término ou eventuais impedimentos, como uma tarefa suspensa.

Exportar SVG. Abre a escolha do destino e exporta o histórico até o tick selecionado, sem avançar o relógio.

Menos zoom, Mais zoom e Ajustar tudo. Alteram somente a escala horizontal do Canvas. Ajustar tudo considera o tempo selecionado e a largura disponível.

Os controles são habilitados conforme o estado da aplicação. Durante a execução em lotes, as intervenções ficam desabilitadas. Após o término, os botões de avanço e execução completa ficam desabilitados.

2.3 Gráfico de Gantt

O gráfico da Figura 4 apresenta duas visões da mesma execução: as CPUs no topo e as tarefas abaixo. Ambas compartilham o eixo horizontal de tempo.

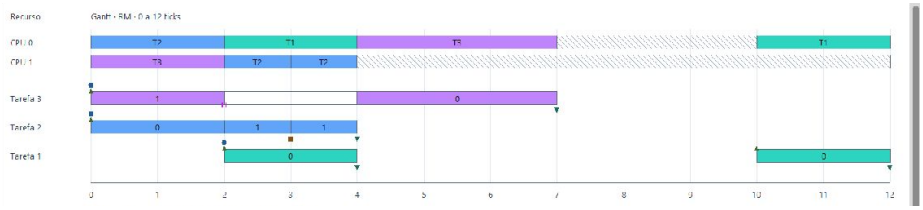


Figura 4: Gráfico de Gantt com faixas de CPU e tarefas no tick 12.

Faixas de CPU. Mostram qual tarefa executou em cada processador. A hachura clara indica que não havia tarefa alocada. A numeração CPU 0, CPU 1 e assim por diante corresponde aos identificadores do motor.

Faixas de tarefas. São organizadas em ordem decrescente de ID, com a tarefa de menor ID mais próxima do eixo do tempo. O número dentro do bloco identifica a CPU usada. O texto é branco ou preto conforme a cor de fundo.

Estado pronta. É representado por um retângulo apenas com contorno. Também é mostrada a espera por CPU logo após a ativação.

Estado suspensa. É representado por um preenchimento pontilhado escuro. A espera pela próxima ativação periódica permanece sem bloco.

Eixo do tempo. Os valores estão em ticks. Um bloco entre a e b representa o intervalo $[a, b)$.

Rolagem. Os rótulos ficam em um Canvas separado: permanecem fixos durante a rolagem horizontal e acompanham o gráfico na vertical.

A função `preparar_gantt` lê o histórico até o instante selecionado. A função `coletar_intervalos`, em `linha_tempo.py`, agrupa execução, espera por CPU, suspensão e ociosidade. A função `desenhar_gantt` transforma esses intervalos em coordenadas, tanto no Canvas quanto no SVG.

2.4 Legenda e eventos

A Figura 5 mostra os símbolos utilizados no gráfico e na exportação.

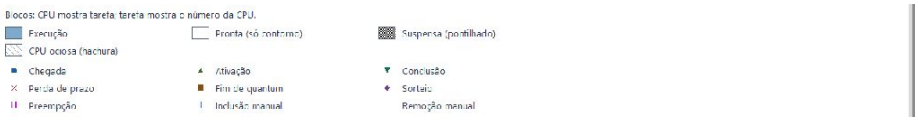


Figura 5: Legenda dos estados e eventos do gráfico de Gantt.

Execução. A cor vem do TCB no histórico. Na faixa da tarefa, o número indica a CPU; na faixa da CPU, o rótulo indica a tarefa. Em trechos muito estreitos, é necessário aproximar o zoom para ler os rótulos.

Contorno, pontilhado e hachura. Indicam, respectivamente, tarefa pronta, tarefa suspensa e CPU ociosa. A espera pela próxima ativação não recebe bloco.

Chegada. É indicada pelo polígono circular azul no instante de ingresso. Sua origem é o evento `chegada`, gerado em `_ativar_tarefas`.

Ativação. É indicada pelo triângulo verde para cima. Corresponde ao evento `ativacao`, produzido para uma ocorrência periódica.

Conclusão. É indicada pelo triângulo verde para baixo. O evento é registrado por `_concluir_ocorrencia` ao terminar uma ocorrência.

Perda de prazo. É indicada por um X vermelho. A origem é a verificação realizada por `_verificar_prazos`. A ocorrência atrasada continua executando; concluir exatamente no prazo é válido.

Fim de quantum. É indicado pelo quadrado marrom. O término da fatia não obriga a tarefa a perder a CPU, caso continue sendo a melhor candidata.

Sorteio. É indicado pelo losango roxo nas tarefas escolhidas após o desempate por sorteio do escalonador.

Preempção. É indicada por duas barras magenta. O evento `preempcao` é registrado em `_despachar` quando uma tarefa que executava deixa de ser selecionada. Migração de CPU e conclusão não geram esse evento.

Inclusão e remoção. São indicadas pelo sinal de adição e pelo traço, respectivamente. O motor registra o instante da intervenção. Os blocos passados de uma tarefa removida continuam visíveis no histórico.

O mapeamento de nomes, cores, posições e formas está em `gantt.EVENTOS` e na função `marcador`. A interface e o SVG utilizam essas mesmas definições.

2.5 Seleção e inspeção da tarefa

A Figura 6 amplia o painel lateral no tick 12. A barra de rolagem permite acessar os eventos exibidos abaixo dos contadores.

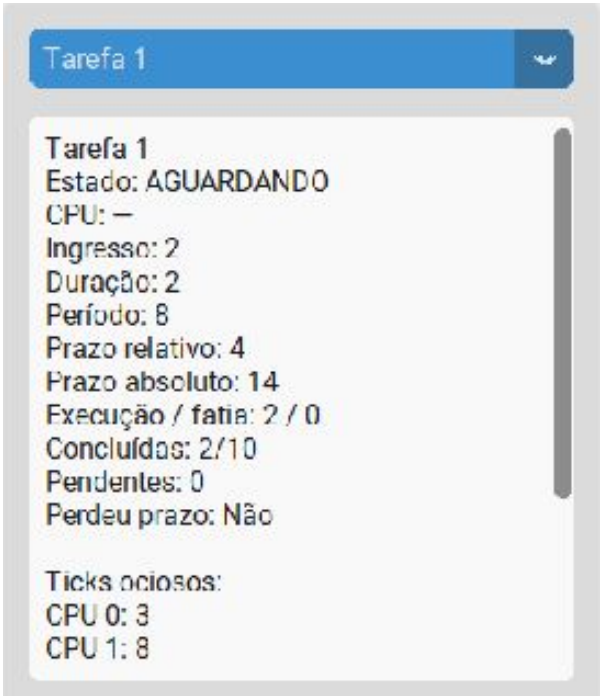


Figura 6: Seletor de tarefa e painel de inspeção.

Seletor e identificação. As opções vêm dos IDs de `estado_atual.tarefas`. A opção selecionada determina o TCB inspecionado.

Estado e CPU. São obtidos de `estado_tarefa.name` e `id_cpu_atual`. O traço indica ausência de CPU alocada.

Ingresso, duração e período. São parâmetros do TCB, provenientes do arquivo de configuração, da inclusão ou de uma edição manual aceita.

Prazo relativo e absoluto. Correspondem a `prazo` e `prazo_absoluto`. O prazo absoluto conta da ativação original, inclusive quando a ocorrência aguardou na fila.

Execução e fatia. Correspondem a `tempo_exec_instancia` e `tempo_exec_quantum`. Enquanto a tarefa aguarda nova ativação, o primeiro contador pode conservar a duração da última ocorrência concluída.

Concluídas e pendentes. Mostram `instancias_concluidas`, de um total de dez, e o tamanho de `ocorrencias_pendentes`. A fila guarda as ativações que aguardam a ocorrência anterior terminar.

Perdeu prazo. Mostra o indicador `perdeu_prazo` da ocorrência representada pelo TCB, apresentado como SIM ou Não.

Ticks ociosos. São os valores de `tempo_desligada` de cada CPU.

Eventos do snapshot. São os dicionários de `estado_atual.eventos`. Eles registram o intervalo que produziu o snapshot e as intervenções manuais realizadas nesse instante.

2.6 Configuração do sistema

A Figura 7 reúne os controles de tarefas e os parâmetros globais. Os campos mostram a configuração do snapshot selecionado. A digitação, por si só, não modifica a simulação.



Figura 7: Ações sobre tarefas e configuração global do sistema.

Ações sobre tarefas. Adicionar abre o formulário de inclusão. Editar utiliza a tarefa selecionada. Remover retira essa tarefa e sua fila, liberando a CPU. As operações são realizadas pelas APIs de `Simulacao`.

Algoritmo. Permite escolher RM ou EDF. O valor inicial vem do arquivo ou de `padroes.ALGORITMO`; a política efetiva pertence ao snapshot.

Quantum. É um inteiro maior ou igual a zero, obtido de `estado_atual.quantum`. Zero remove o limite de fatia, mantendo a preempção; valores positivos permitem registrar os fins de fatia.

Quantidade de CPUs. É um inteiro maior ou igual a um, obtido do tamanho de `estado_atual.cpus`. Reduzir a quantidade libera as tarefas das CPUs retiradas.

Aplicar configuração. Converte os campos e chama `configurar_sistema`. O motor valida os valores antes de aceitar. Uma alteração aceita descarta apenas o futuro do instante selecionado.

Sugerir padrões. Preenche RM, quantum 0 e duas CPUs a partir de `padroes.py`. O usuário pode substituir as sugestões antes de aplicar.

3 Formulários de tarefas

3.1 Inclusão de tarefa

A Figura 8 apresenta o formulário aberto pelo botão Adicionar.



Figura 8: Formulário de inclusão de uma tarefa.

ID. A sugestão é o próximo inteiro positivo disponível no histórico selecionado. O campo é editável na inclusão, mas IDs já utilizados nesse histórico não podem ser reutilizados.

Ingresso. A sugestão é o tick atual. O motor rejeita uma chegada anterior ao instante selecionado.

Duração, período e prazo relativo. As sugestões são, respectivamente, 1, 10 e 10 ticks. Os valores devem ser inteiros positivos e podem ser alterados antes de salvar.

Cor RGB. É sugerida uma cor disponível da paleta. Se necessário, são geradas novas cores RGB. O valor deve ter seis dígitos hexadecimais e ser distinto das cores das tarefas presentes.

Estado. Uma nova tarefa começa em AGUARDANDO. Progresso, CPU, contadores e prazo absoluto são inicializados pelo motor.

Texto de orientação. Explica que o ingresso deve ocorrer no tick atual ou em um instante posterior.

Sugerir padrões. Repõe as sugestões produzidas por `Simulacao.sugerir_tarefa` e `padroes.py`.

Salvar. Envia os campos a `adicionar_tarefa`. O motor valida a operação, e os erros são apresentados no próprio formulário.

Cancelar e fechar. Cancelar ou o botão de fechamento da janela encerram o formulário sem incluir uma tarefa.

3.2 Edição de tarefa

O formulário da Figura 9 recebe os dados do TCB selecionado. O ID fica bloqueado. Ingresso, duração, período, prazo e cor mantêm os significados da tela de inclusão. Salvar envia somente os campos alterados ao motor.

O seletor de estado permite solicitar as seguintes transições, sujeitas à validação:

AGUARDANDO. Abandona a ocorrência ativa e sua fila, sem acrescentar conclusões.

PRONTA. Preserva uma ocorrência ativa. Partindo de espera, inicia uma ocorrência no tick atual, respeitando o instante de ingresso.

EXECUTANDO. Preserva uma ocorrência ativa e exige uma CPU própria ou livre. Partindo de espera, também inicia uma ocorrência no instante atual.

SUSPENSA. Preserva a ocorrência ativa e seu progresso, liberando a CPU. A execução completa exige retomada manual. Partindo de espera, inicia uma ocorrência no tick atual, se essa ativação for válida.

FINALIZADA. Só é aceita após dez conclusões. Uma tarefa que já concluiu dez ocorrências não pode ser reativada.

Na edição, Sugerir padrões preserva ID e estado e sugere parâmetros compatíveis com o progresso. Cancelar e o fechamento da janela não aplicam mudanças. A advertência informa que uma edição aceita descarta o futuro. As regras são implementadas por `modificar_tarefa_em_execucao` e seus auxiliares.



Figura 9: Formulário de edição de uma tarefa existente.

4 Mensagens e validação

As Figuras 10 e 11 apresentam dois exemplos de rejeição. Valores inválidos não são aplicados ao estado da simulação.

Mensagem no formulário. Ao informar duração zero, o motor rejeita o valor e a interface apresenta o motivo em vermelho, permitindo a correção.

Mensagem no rodapé. Quantidade de CPUs zero é rejeitada. O campo pode manter o texto digitado, enquanto o motor conserva a configuração válida anterior.

Arquivo inválido e avisos. O parser informa o motivo e a linha do erro. Campos vazios recebem padrões com aviso; tarefas aperiódicas são ignoradas com aviso. Prazo zero é aceito em uma aperiódica ignorada no Projeto A; tarefas periódicas exigem prazo positivo.

Execução e exportação. Uma tarefa suspensa impede a execução completa, e a interface informa o motivo. Falhas de escrita preservam os resultados para uma nova tentativa de exportação.

A GUI apresenta erros de valor, execução e entrada ou saída. As exceções correspondentes são `ValueError`, `RuntimeError` e `OSError`. As regras básicas estão em `validacao.py`. O motor verifica as consequências para a ocorrência, a fila e a CPU.

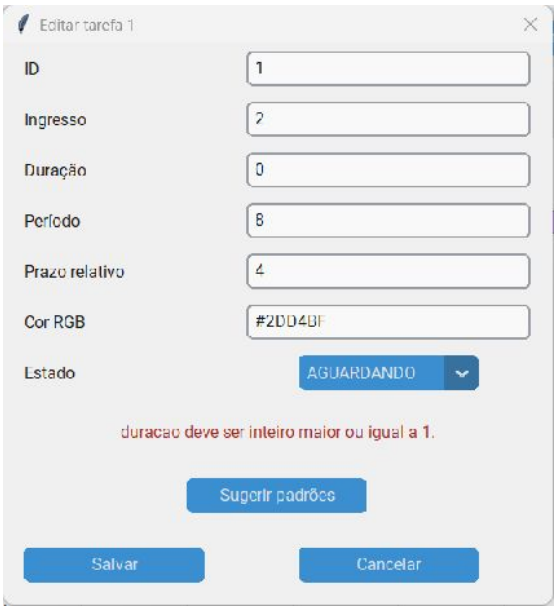


Figura 10: Rejeição de uma duração igual a zero no formulário de edição.

Figura 11: Mensagem no rodapé após informar quantidade de CPUs igual a zero.

5 Diálogos de arquivos

5.1 Seleção da configuração

O diálogo Abrir da Figura 12 é fornecido pelo Windows e chamado por `tkinter.filedialog`. A pasta mostrada na captura é um exemplo de navegação, não um caminho obrigatório.

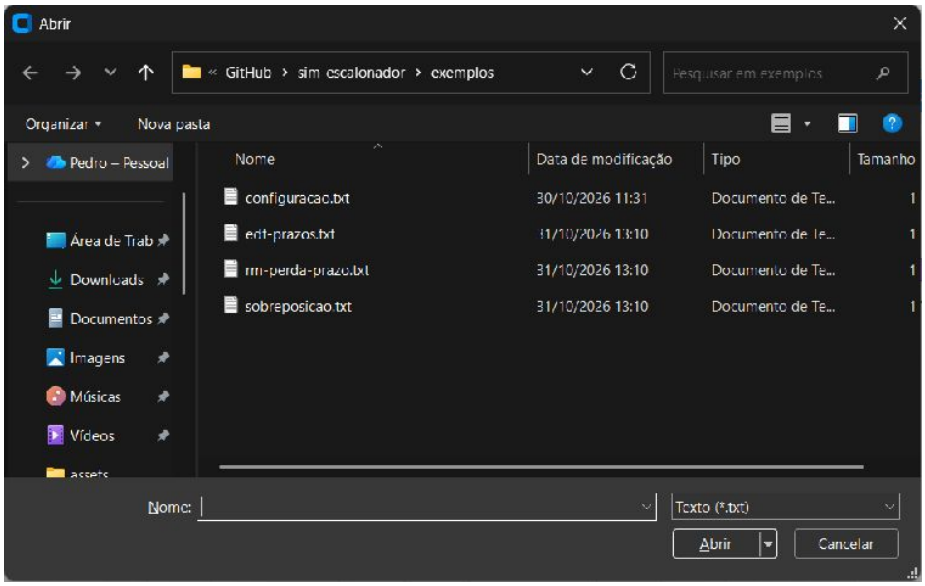


Figura 12: Diálogo de seleção de um arquivo de configuração.

Navegação e endereço. Mostram a pasta atual, os comandos voltar, avançar e subir e os atalhos laterais. São recursos do Windows.

Busca e ferramentas. Pesquisar, organizar, criar pasta, alterar o modo de exibição e obter ajuda são controles nativos do seletor.

Lista e colunas. Nome, data, tipo e tamanho são metadados fornecidos pelo sistema de arquivos, não pelo motor da simulação.

Nome e tipo. Permitem escolher um nome ou caminho e filtrar os arquivos. O filtro inicial é Texto, com extensão `.txt`, e existe a opção Todos. As opções são definidas em `gui.abrir`.

Abrir. Devolve o caminho ao método `carregar_arquivo`. O parser atua somente depois da escolha.

Cancelar e fechar. Encerram o diálogo sem substituir a configuração.

5.2 Seleção do destino SVG

O diálogo Salvar como, apresentado na Figura 13, também é fornecido pelo Windows. A pasta da captura foi usada apenas para documentar a operação; o usuário pode selecionar outro destino.

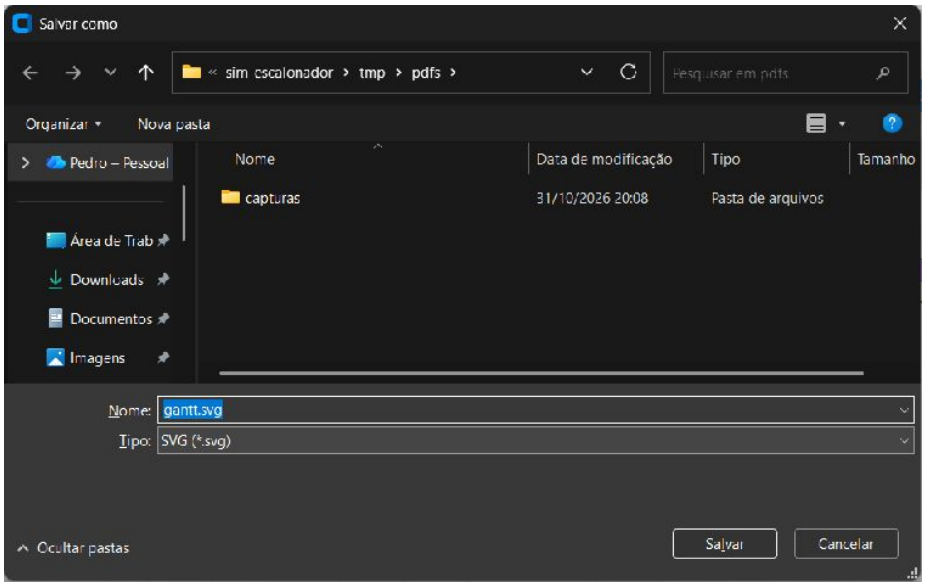


Figura 13: Diálogo de seleção do destino da exportação SVG.

Navegação e lista. Mostram a pasta de destino e seu conteúdo, fornecidos pelo sistema de arquivos. Os atalhos, busca, visualização e criação de pasta são recursos do Windows.

Nome e tipo. O nome `gantt.svg` é sugerido por `padroes.NOME_SVG`. O filtro e a extensão padrão são SVG.

Salvar. Devolve o caminho ao método `exportar_arquivo`. O Windows pode solicitar confirmação se o destino já existir.

Cancelar e fechar. Encerram o diálogo sem gravar o arquivo.

A função `gantt.exportar_svg` utiliza todo o histórico até o instante selecionado, com legenda e escala própria. O zoom e a rolagem da interface não recortam o SVG. Depois da gravação, o rodapé informa o destino. A escolha do arquivo não altera o estado da simulação nem salva uma configuração.

6 Resultado final e gravação automática

O exemplo da Figura 14 termina no tick 185, com dez ocorrências concluídas para cada uma das três tarefas. O painel mostra a Tarefa 1 como FINALIZADA e sem CPU. Os campos preservam os dados da última ocorrência, e o Gantt conserva a execução completa.

Ao concluir pela interface com um arquivo de entrada, o método `salvar_final` grava um SVG ao lado da configuração. Um sufixo numérico é acrescentado quando o nome já existe. Sem arquivo de entrada, a interface solicita um destino. Cancelamento ou falha de escrita preservam o resultado. O botão Exportar SVG também permite salvar um estado intermediário.

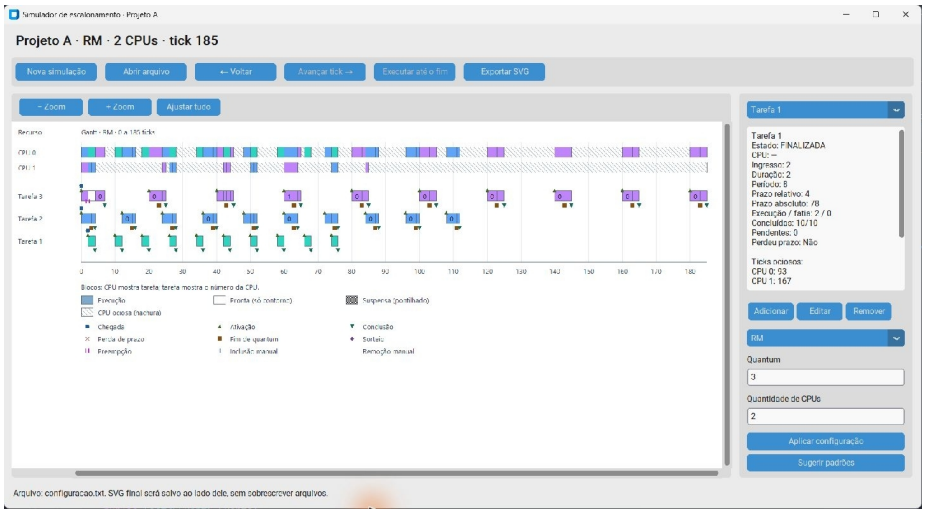


Figura 14: Resultado final da configuração de exemplo no tick 185.

7 Origem das informações no código

Entrada e sugestões. `parser.py` converte o cabeçalho e os registros de tarefas. `padroes.py` fornece os valores sugeridos.

Dados do sistema. As estruturas estão em `models.py`: TCB, `Ocorrencia`, `Cpu` e `EstadoSimulacao`. A interface consulta o snapshot selecionado pela propriedade `estado_atual` da simulação.

Evolução e intervenções. `engine.py` ativa, despacha, executa e verifica prazos. Também mantém o histórico e valida configuração, inclusão, remoção e edição.

Prioridades e sorteios. `scheduler.py` ordena as candidatas por RM ou EDF e aplica os desempates. O motor registra os sorteios e as tarefas selecionadas nos eventos.

Intervalos. O módulo `linha_tempo.py` agrupa os trechos do histórico. O módulo `gantt.py` desenha intervalos, CPUs e símbolos no Canvas e no SVG produzido por drawsvg.

Janelas e mensagens. `gui.py` monta `JanelaSimulador` e `DialogoTarefa`, lê formulários e apresenta os resultados do motor. `validacao.py` valida inteiros, limites, cores RGB e algoritmo.

A configuração usada nas figuras de simulação é:

```
RM;3;2
1;2DD4BF;2;2;8;4;
2;60A5FA;0;4;12;8;
3;C084FC;0;5;20;12;
```